



**NOVA**  
*an OpenStack Community Project*

# CAHIER DE CONCEPTION

**Automatisation et optimisation du provisionnement des machines avec Nova**

Rédigé par :

MEMEZAGUE NGUEMO Fabiola  
YAKAM TCHAMOU Rick Vadel

Examineur:

M. NGUIMBUS 1

# PLAN



- I. INTRODUCTION
- II. CONTEXTE DU PROJET ET OBJECTIF
- III. ARCHITECTURE DU SYSTÈME
- IV. DIAGRAMME DE SÉQUENCE
- V. DIAGRAMME DE CLASSE
- VI. DIAGRAMME DE COMPOSANTS
- VII. DIAGRAMME DE DÉPLOIEMENT
- VIII. PATRON DE CONCEPTION
- IX. CONCLUSION

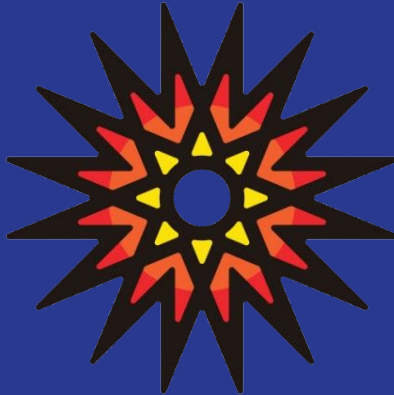
---

# INTRODUCTION

Après l'identification des besoins fonctionnels et non fonctionnels dans le cahier d'analyse, le présent cahier de conception vise à traduire ces besoins en une architecture technique cohérente et exploitable.

Ce document décrit la conception du système d'extension du module Nova d'OpenStack, intégrant un mécanisme de facturation basé sur l'usage réel des ressources et un système de scaling automatique (scale up / scale down). La modélisation repose sur le langage UML afin de garantir une représentation claire des interactions, des structures internes et du déploiement de la solution.

# CONCEPTION DU PROJET ET OBJECTIFS



# CONTEXTE DU PROJET ET OBJECTIFS



## CONTEXTE

OpenStack est une plateforme de cloud computing open source permettant la gestion des ressources virtualisées (calcul, réseau, stockage ...). Parmi ses composants, Nova (Compute) joue un rôle central en assurant le provisionnement, la gestion du cycle de vie et le scheduling des machines virtuelles.

Cependant, bien que Nova propose des fonctionnalités robustes pour la gestion des instances, il ne fournit pas nativement :

- un système de facturation automatique basé sur la consommation réelle des ressources.
- un mécanisme avancé de scaling automatique orienté utilisateur.

Ces limitations entraînent une sous-utilisation des ressources, une absence de visibilité sur les coûts et une difficulté à adapter dynamiquement les capacités des machines virtuelles.

# CONTEXTE DU PROJET ET OBJECTIFS



## OBJECTIFS

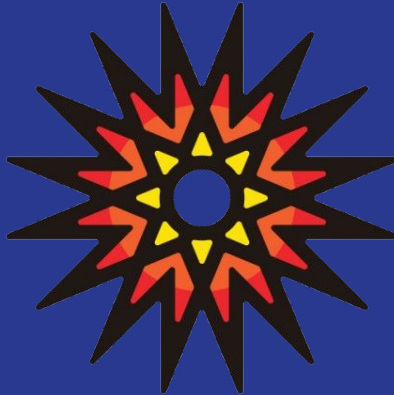
L'objectif principal du projet est de concevoir une extension fonctionnelle du module Nova afin de :

- Automatiser la gestion des coûts liés à l'utilisation des ressources cloud.
- Optimiser l'allocation des ressources grâce à un mécanisme de scaling automatique.

Objectifs spécifiques :

- Concevoir un module de facturation basée sur les métriques collectées (CPU, RAM, stockage,...)
- Concevoir un système de scale up et scale down automatique déclenchée par des seuils configurables.
- Intégrer ces fonctionnalités dans l'écosystème OpenStack existant.

# ARCHITECTURE DU SYSTÈME



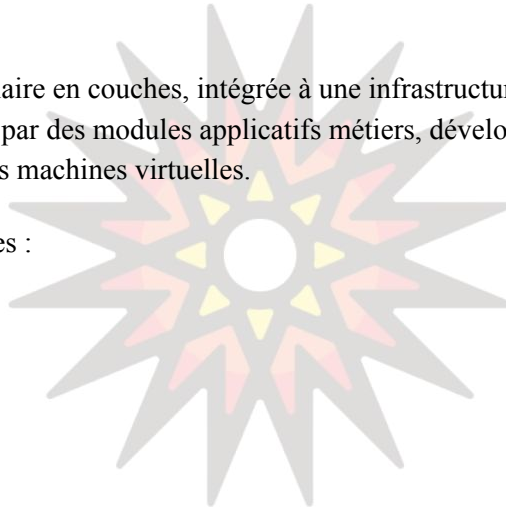
# ARCHITECTURE DU SYSTÈME

## VUE GLOBAL

Le projet repose sur une architecture modulaire en couches, intégrée à une infrastructure OpenStack existante. Il ne modifie pas les services natifs, mais les étend par des modules applicatifs métiers, développés spécifiquement pour répondre aux besoins de facturation et de mise à l'échelle automatique des machines virtuelles.

L'architecture finale repose sur cinq grandes couches :

1. La couche Présentation
2. La couche Authentification et Sécurité
3. La couche Métier (Modules Nova étendus)
4. La couche Services OpenStack
5. La couche Données





# ARCHITECTURE DU SYSTÈME

## COUCHE PRÉSENTATION

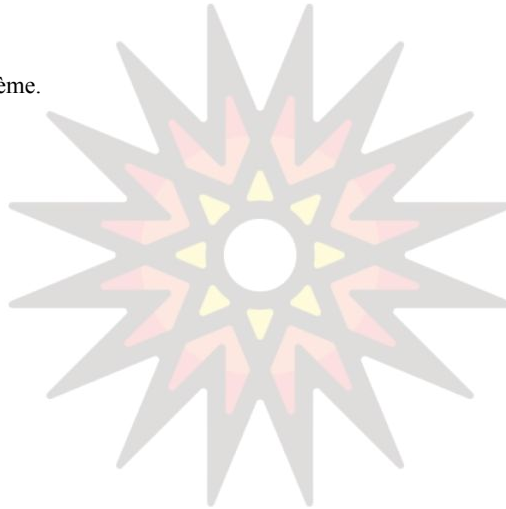
Cette couche assure l'interaction entre les utilisateurs et le système.

### Composants

- Interface Web (application dédiée )
- API REST du système

### Fonctions principales

- Connexion / déconnexion des utilisateurs
- Création et gestion des machines virtuelles
- Définition des seuils de scaling
- Consultation de la consommation des ressources
- Consultation des factures et reçus de paiement



# ARCHITECTURE DU SYSTÈME

## COUCHE AUTHENTIFICATION ET SÉCURITÉ

Garantir un accès sécurisé au système et contrôler les droits des utilisateurs.

### Composants

- Module d'authentification applicatif
- Keystone



# ARCHITECTURE DU SYSTÈME

## COUCHE MÉTIER

- Module de facturation

Garantir un accès sécurisé au système et contrôler les droits des utilisateurs.

### Composants

- Module d'authentification applicatif
- Keystone  
Assurer la facturation des ressources consommées par les machines virtuelles.

### Fonctions

- Collecte des métriques de consommation
- Calcul des coûts (CPU, RAM, stockage, durée)
- Génération des factures
- Gestion du solde utilisateur
- Suspension ou alerte en cas de solde insuffisant

### Interactions

- Ceilometer (collecte des métriques)
- Gnocchi (stockage et agrégation)
- Nova (suspension, arrêt ou gestion des VM)
- 

### Fonctionnement

- Le module applicatif reçoit les identifiants utilisateur
- Il s'appuie sur **Keystone** pour authentifier les utilisateurs et récupérer un token
- Le token est utilisé pour autoriser l'accès aux services OpenStack

# ARCHITECTURE DU SYSTÈME

## COUCHE MÉTIER

- Module de scaling

Adapter dynamiquement les ressources des machines virtuelles selon la charge.

### Fonctions

- Analyse des métriques d'utilisation
- Comparaison avec les seuils configurés
- Déclenchement des actions de scale up / scale down

### Interactions

- Ceilometer (récupération des métriques)
- Gnocchi (historisation)
- Nova (resize, start/stop, allocation de ressources)

# ARCHITECTURE DU SYSTÈME



## COUCHE OPENSTACK

Cette couche regroupe les **services natifs OpenStack**, utilisés sans modification.

### Services utilisés

- **Nova** : gestion du cycle de vie des machines virtuelles
- **Keystone** : authentification et gestion des rôles
- **Ceilometer** : collecte des métriques de consommation
- **Gnocchi** : stockage et agrégation des métriques

Les modules développés communiquent avec ces services via leurs **API officielles**.

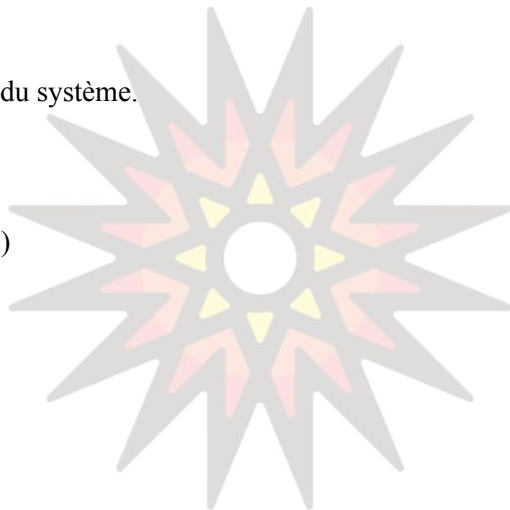
# ARCHITECTURE DU SYSTÈME

## COUCHE OPENSTACK

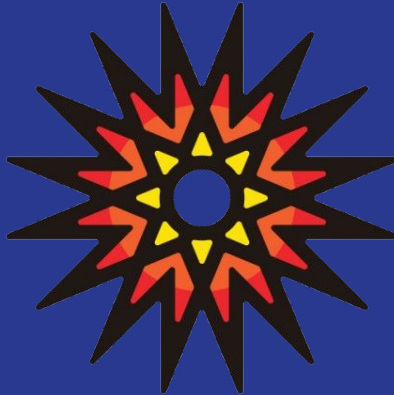
Stocker les données nécessaires au fonctionnement du système.

### Bases de données

- Base OpenStack existante (Nova, Keystone)
- Base de données des modules ajoutés

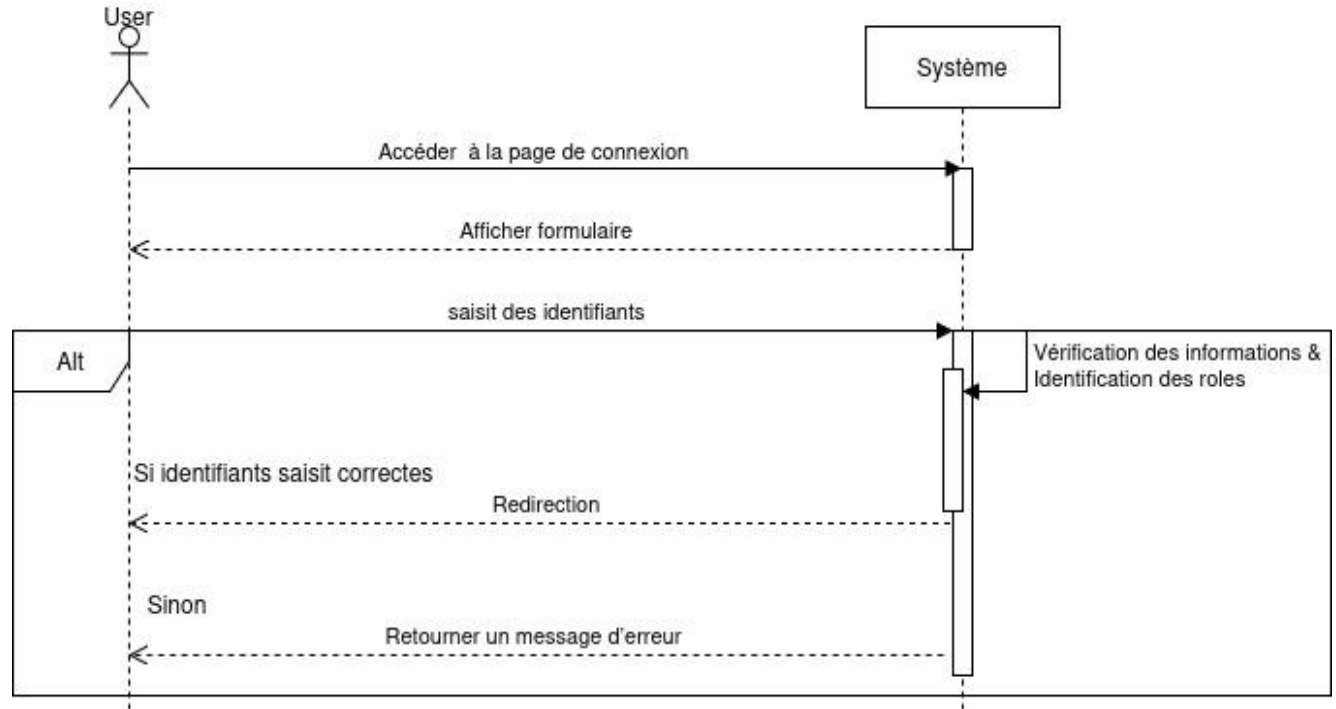


# PRÉSENTATION DES DIAGRAMMES



# DIAGRAMME DE SÉQUENCE

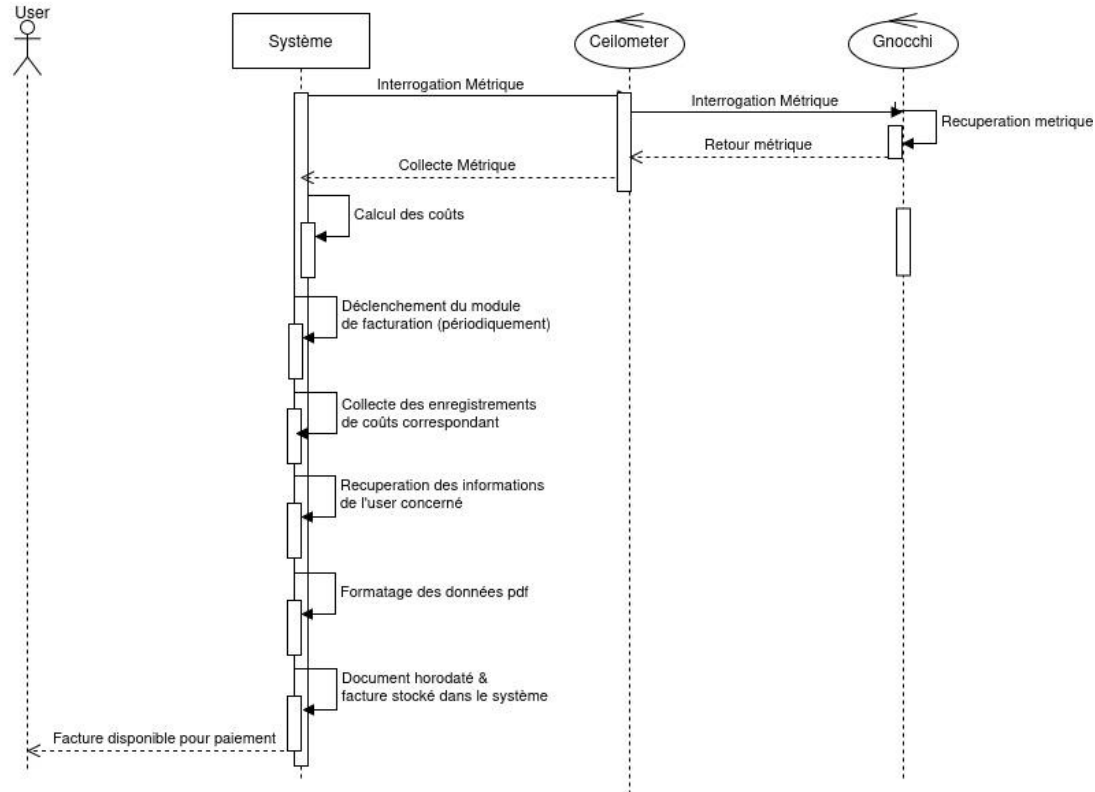
Authentification





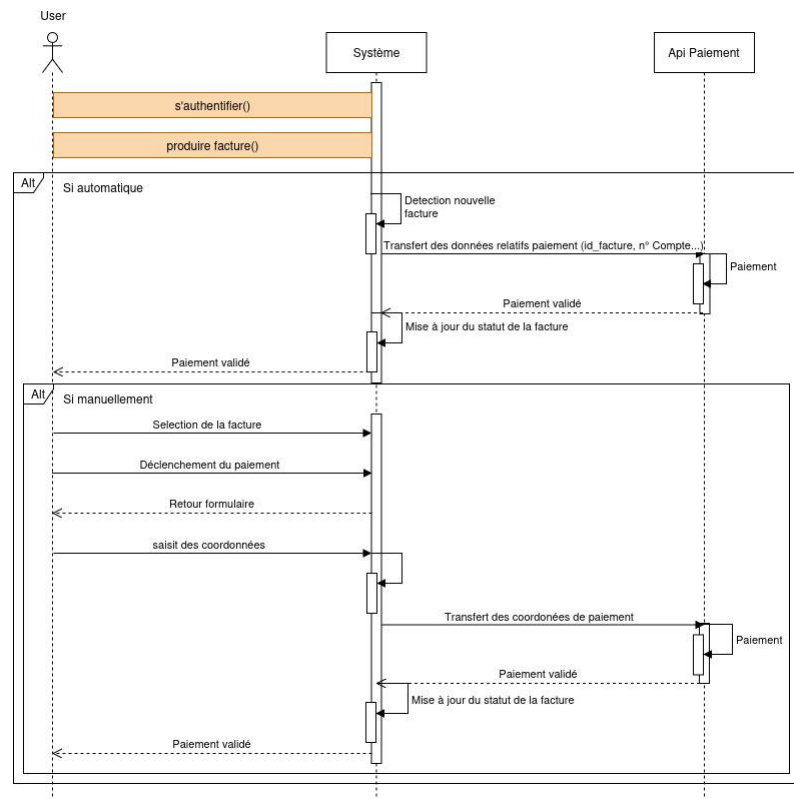
# DIAGRAMME DE SÉQUENCE

Génération de facture



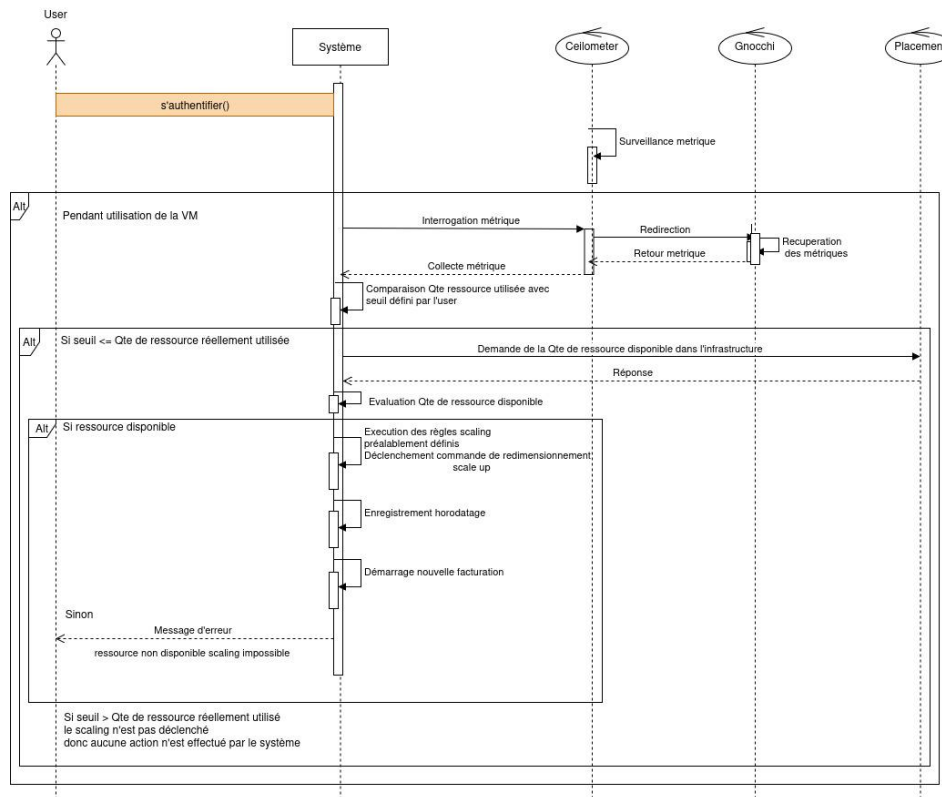
# DIAGRAMME DE SÉQUENCE

Paieement facture



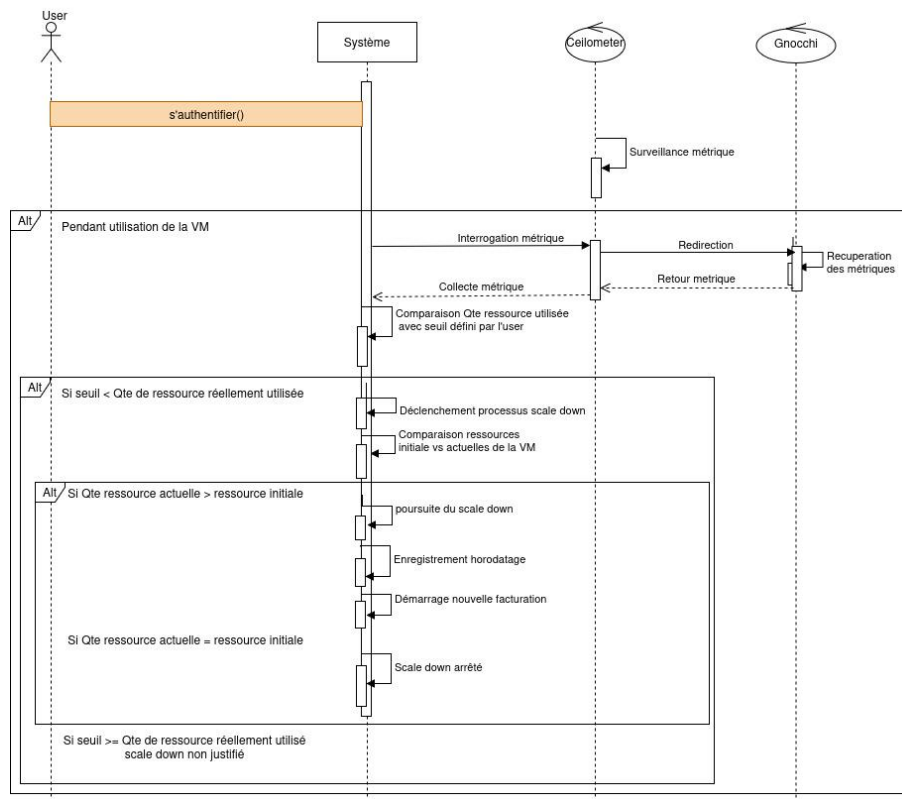
# DIAGRAMME DE SÉQUENCE

Scale up

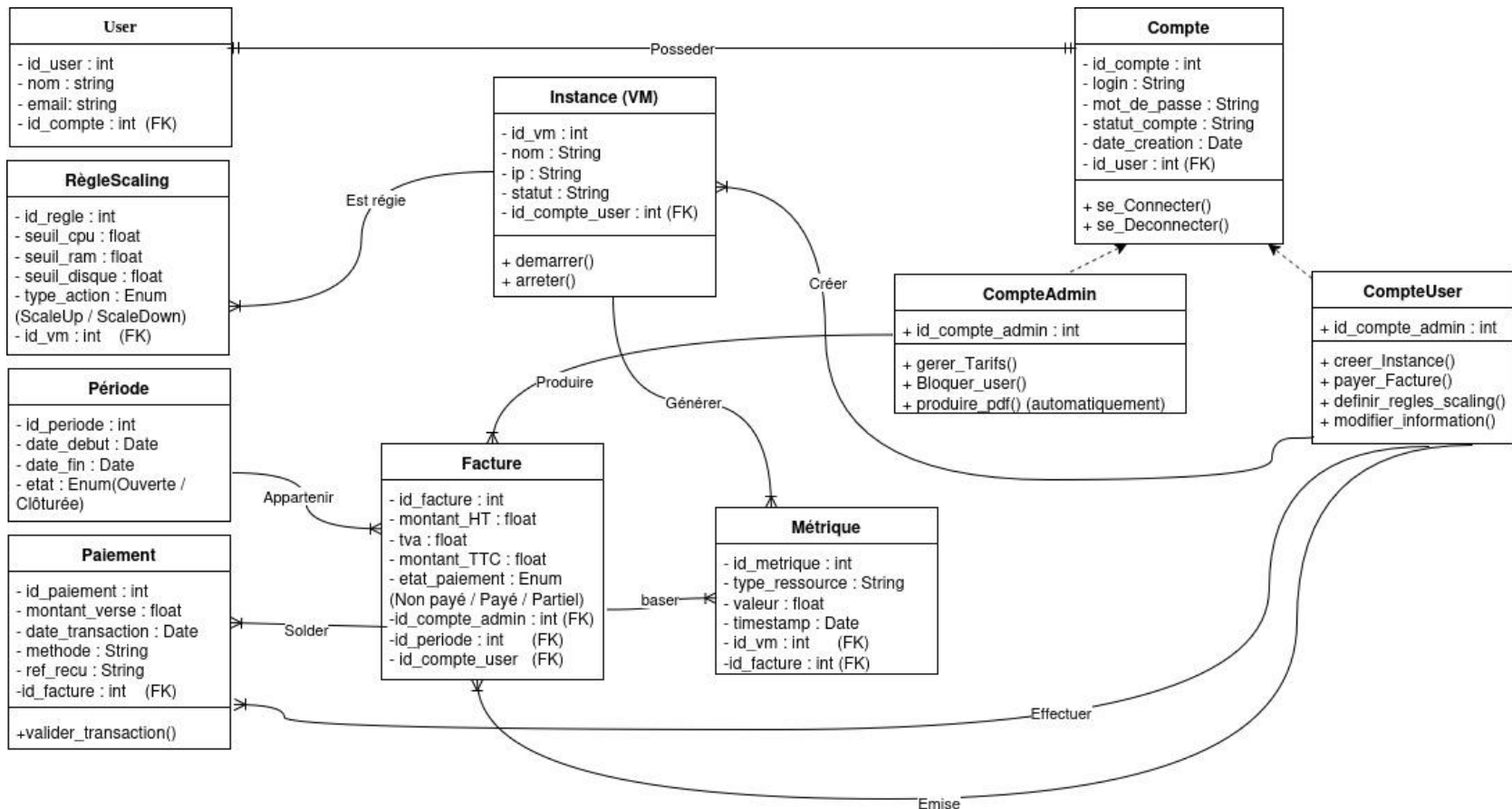


# DIAGRAMME DE SÉQUENCE

Scale down



# DIAGRAMME DE CLASSE



# PATRON DE CONCEPTION

Pour répondre aux besoins d'automatisation, de monitoring et de facturation sans alourdir le noyau d'OpenStack Nova, le projet s'appuie sur une combinaison de patron : le patron de conception **Observateur et stratégie**.

## 1. Le Patron Observateur

Le choix de ce patron est justifié par la nécessité de surveiller en temps réel l'état des machines virtuelles (CPU, RAM, Uptime) pour déclencher des actions automatiques (scaling, calcul de coût).

- **Le Sujet (Subject)** : Le service **Ceilometer/Gnocchi**. Il détient l'état des ressources et les métriques de performance.
- **Les Observateurs (Observers)** : Le **Module d'Auto-scaling** : Il "observe" les seuils de charge. Dès qu'une métrique dépasse la limite définie, il réagit en envoyant un ordre de redimensionnement.
- **Le Module de Facturation** : Il "observe" la consommation et l'uptime pour mettre à jour les coûts en temps réel ou générer des factures périodiques.

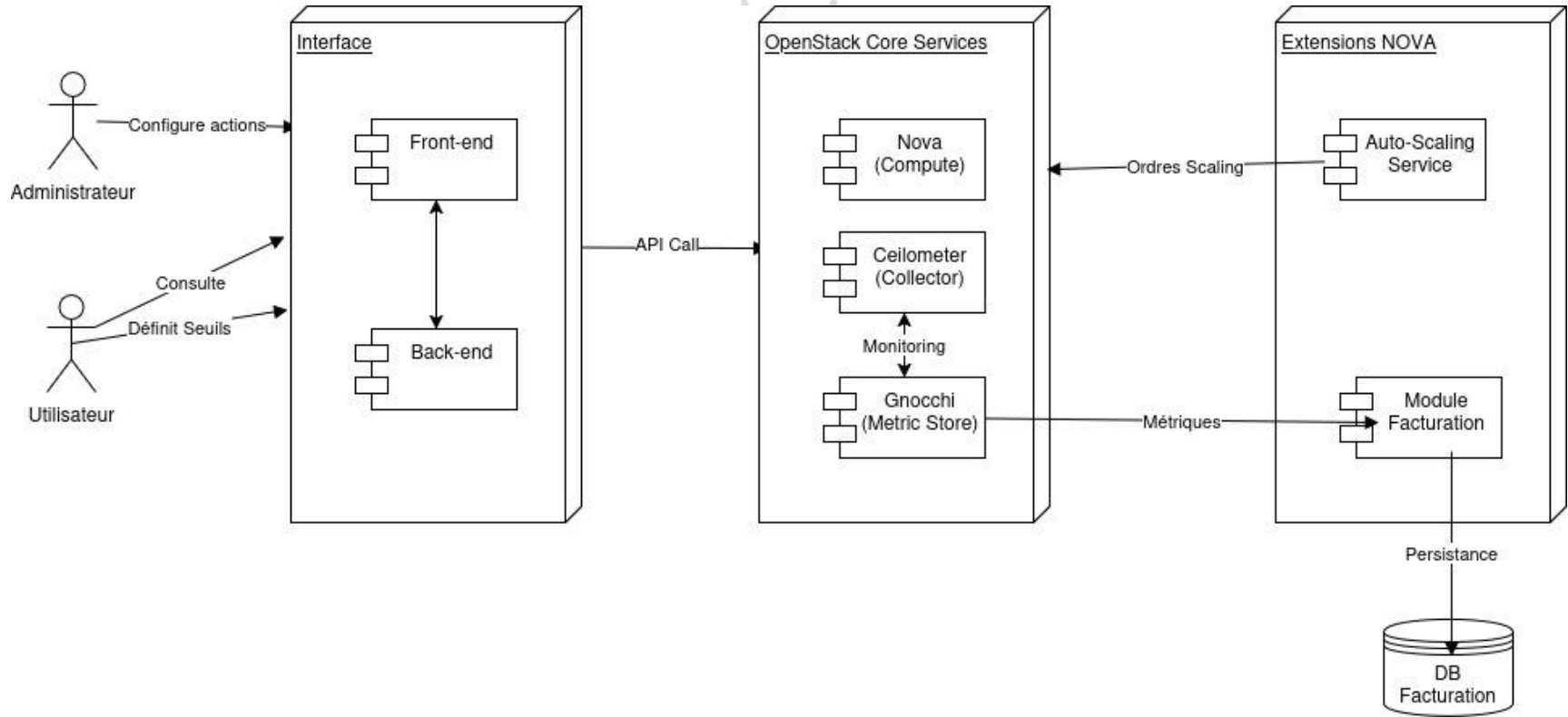
## 2. Patron Stratégie

Pour la partie facturation, nous utilisons le patron **Stratégie**. Il permet d'appliquer différentes méthodes de calcul de coût (ex: tarif fixe par instance, tarif à la consommation réelle, ou tarif dégressif) sans modifier le code principal du module.

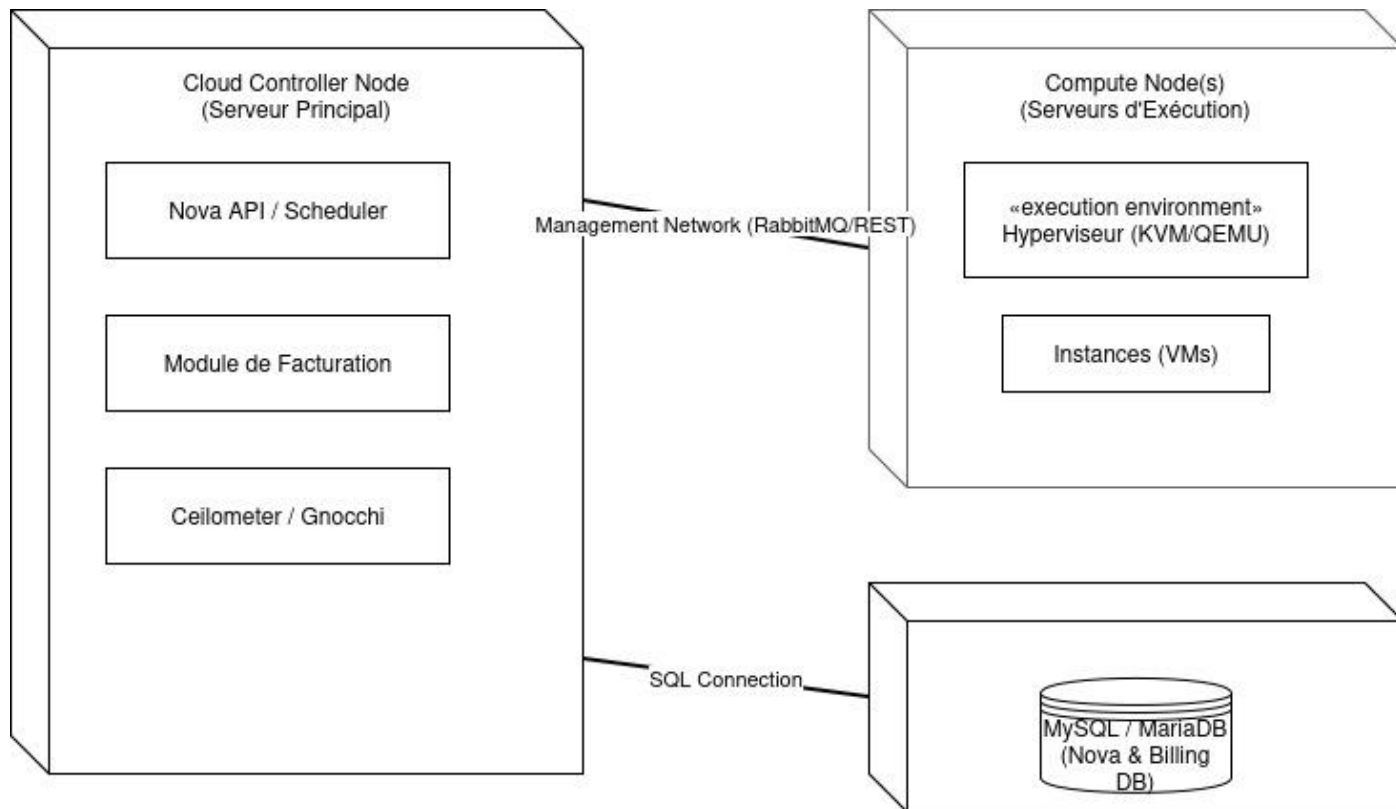
## 3. Architecture Modulaire et Faible Couplage

Le projet applique également le principe de **faible couplage**. Vos modules (Scaling et Facturation) sont des entités indépendantes qui communiquent avec Nova via des **API REST**.

# DIAGRAMME DES COMPOSANTS



# DIAGRAMME DE DÉPLOIEMENT





# CONCLUSION

Ce cahier de conception propose une modélisation complète et structurée de l'extension du module Nova d'OpenStack intégrant la facturation et le scaling automatique. Il constitue une base solide pour la phase de développement et d'implémentation.

La conception retenue garantit une meilleure maîtrise des ressources, une optimisation des coûts et une amélioration significative de l'expérience utilisateur dans l'exploitation du cloud OpenStack.

